

## **Unit 1: Introduction to Python Programming**

**Topic:** Introduction to Python, Variables in Python, Data Types in Python, Conditional Statements, Loops in Python, Lists, Tuples, Sets, and Dictionaries.

---

### **1.Introduction to Python**

#### **1.1 What is Python?**

Python is a high-level, interpreted, and general-purpose programming language created by Guido van Rossum in 1991. It is known for its simple and readable syntax, making it easy for beginners to learn and use. Python is widely used to develop websites, applications, software, and for data analysis, and is suitable for both students and professionals.

#### **1.2 Features of Python**

- Easy to learn and use
- Open source and free
- Platform independent
- Object-Oriented language
- Large standard library
- Used in AI, Data Science, Web Development, Automation, etc.

#### **1.3 Applications of Python**

- Web Development (Django, Flask)
- Data Science & Machine Learning
- Instagram uses Python
- YouTube uses Python
- Software Development
- Automation/Scripting
- Game Development
- Networking

#### **1.4 First Python Program**

```
print("Hello World")
```

## 2. What is Operator?

Ans. Operators are symbols that tell Python to perform actions like addition, comparison, or logical checking.

- **Types of Operators**

- a. **Arithmetic Operators:**

Used for mathematical calculations. ( +, -, \*, /, %, \*\*)

- b. **Relational (Comparison) Operators:**

Used to compare two values. ( ==, !=, >, <, >=, <=)

- c. **Logical Operators:**

Used to combine conditions.( and , or, not )

- d. **Assignment Operators:**

Used to assign values.( =, +=, -=, \*=, /=, %=)

### 2.1 Input in Python

input() : input() statement is used to accept values (using keyboard) from user.

- input() : result for input() is always a str.
- int(input()): int
- float(input()): float

## 3. What is Keywords?

Ans. Python has a set of keywords that are reserved words that cannot be used as variable names, function names, or any other identifiers:

Below are the Keywords:

False, None, True, and, as, assert, break, class, continue, def, del, elif, else, except, finally, for, from, global, if, import, in, is, lambda, nonlocal, not, or, pass, raise, return, try, while, with, yield

## 4. What is Comments ?

Ans. Comments are notes written inside a program to explain what the code is doing.

### 4.1 Types of Comments in Python:

- **Single line comments:** Starts with #
- **Multi-Line Comment:** Python does not have a special symbol, but we use triple quotes.

## 5. Basic Programming Concepts

Every program follows three basic steps:

### ❖ Input

- Taking data from the user.  
`name = input("Enter your name: ")`

### ❖ Process

- Performing operations on the input.  
`marks = int(input("Enter marks: "))`  
`result = marks + 5`

### ❖ Output

- Displaying the result.  
`print(result)`

## 6. Variables in Python

### Definition

A variable is a name given to a memory location used to store data.

### ❖ Rules for Naming Variables

- Must start with a letter or underscore
- Cannot start with a number
- Cannot use keywords
- Case-sensitive

### Examples:

```
a = 10
name = "Rahul"
price = 99.5
```

- Python automatically decides the type of variable.

## 7. Data Types in Python

Python supports different types of data:

### 7.1 Integer (int) : Stores whole numbers.

Example:

```
x = 10
```

```
y = 20
```

### 7.2 Float (float) : Stores decimal numbers.

Example:

```
y = 3.14
```

```
x = 4.53
```

### 7.3. Boolean(bool): Stores True or False.

Example:

```
is_pass = True
```

### 7.8 Type Checking: Used to check which type of data is stored in a given variable.

Example:

```
x = 10
```

```
print(type(x))
```

## 8. What is a String?

Ans. A string in Python is a sequence of characters enclosed in single quotes, double quotes. It is used to store and manipulate text data.

Example:

```
name = "Shiv"
```

```
message = 'Hello World'
```

➤ Here, "Shiv" and 'Hello World' are strings.

### 8.1 How to Create a String?

Ans. You can create a string using:

Single quotes -- 'Hello'

Double quotes -- "Hello"

**Example:**

```
text1 = 'Python'  
text2 = "Programming"  
text3 = "This is a multi-line string"
```

**8.2 Accessing Characters (Indexing):-** Each character in a string has a position (index).**Example:**

```
word = "Python"  
print(word[0]) # P  
print(word[3]) # h
```

Index starts from 0

**8.1 String Length:-** Use len() to find the number of characters.**Example:**

```
name = "Prince"  
print(len(name)) # 6
```

**8.2 String Concatenation (Joining Strings)****Example:**

```
first = "Shiv"  
last = "Kumar"  
print(first + " " + last)
```

**Output:** Shiv Kumar

**8.3 Strings are Immutable:**

Once created, a string cannot be changed directly.

**Example:**

```
name = "Prince"  
name[0] = "p" Error
```

- You must create a new string instead.

## 8.4 Important String Methods

- **upper()** – Convert to capital letters

**Example:**

```
text = "hello"  
print(text.upper())
```

**Output:** HELLO

- **lower()** – Convert to small letters

**Example:**

```
text = "HELLO"  
print(text.lower())
```

**Output:** hello

- **len()** – Find length of string

**Example:**

```
text = "Python"  
print(len(text))
```

**Output:** 6

- **count()** – Count a character

**Example:**

```
text = "banana"  
print(text.count('a'))
```

**Output:** 3

- **find()** – Find position of character/word

**Example:**

```
text = "Hello World"  
print(text.find("World"))
```

**Output:** 6

- **replace()** – Replace word

**Example:**

```
text = "I like Java"
print(text.replace("Java", "Python"))
```

**Output:** I like Python

- **strip()** – Remove extra spaces

**Example:**

```
text = " hello "
print(text.strip())
```

**Output:** hello

- **split()** – Convert string to list

**Example:**

```
text = "apple,banana,mango"
print(text.split(","))
```

**Output:** ['apple', 'banana', 'mango']

## 9. Conditional Statements: Used for decision making.

### 9.1 if Statement

**Example:**

```
age = 20
if age >= 18:
    print("Eligible to vote")
```

### 9.2 if-else Statement

**Example:**

```
marks = 35
if marks >= 40:
    print("Pass")
else:
    print("Fail")
```

### 9.3 if-elif-else Statement

Example:

```
marks = 75

if marks >= 90:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

**10. Loops in Python:** Loops are used to repeat tasks.

**10.1 for Loop:** Used when number of repetitions is known.

Example:

```
for i in range(1,6):
    print(i)
```

**10.2 while Loop:** Used when repetitions depend on condition.

Example:

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

## 11 . Lists

**Definition:** A list is an ordered collection of items.

### Features

- Changeable (mutable)
- Allows duplicate values
- Uses square brackets []

### Example

```
numbers = [10, 20, 30, 40]
print(numbers[0])
```

### Operations on List

```
numbers.append(50)
numbers.remove(20)
```



## 12. Tuples

**Definition:** A tuple is an ordered collection that cannot be changed (immutable).

### Features

- Fixed size
- Faster than list
- Uses ()

### Example

```
t = (1, 2, 3, 4)
print(t[1])
```

## 13. Sets

**Definition:** A set is an unordered collection of unique elements.

### Features

- No duplicates allowed
- No indexing
- Uses {}

### Example

```
s = {1, 2, 3, 3, 4}
print(s)
Output: {1, 2, 3, 4}
```

## 14. Dictionaries

**Definition:** A dictionary stores data in key-value pairs.

### Features

- Mutable
- Fast access using keys
- Uses {}

### Example

```
student = {
    "name": "Aman",
    "age": 21,
    "branch": "CSE"
}

print(student["name"])
```

## 15. Difference Between List, Tuple, Set, Dictionary

| Feature    | List | Tuple | Set | Dictionary         |
|------------|------|-------|-----|--------------------|
| Ordered    | Yes  | Yes   | No  | Yes                |
| Mutable    | Yes  | No    | Yes | Yes                |
| Duplicates | Yes  | Yes   | No  | Keys not duplicate |
| Syntax     | []   | ()    | {}  | {key:value}        |

### Programming Questions

- Program to check even/odd.
- Program to find sum using loop.
- Program using list operations.
- Program to store student data in dictionary.

\*\*\*\*\*

## Unit 2: Functions

Topics: Functions, OOP Concepts, Exception, Handling, File Handling

Prepared by: Prince Kumar, Assistant Professor, CSE

---

### 1.1 What is a Function?

A function is a block of code that performs a specific task and can be used again and again in a program. Instead of writing the same code many times, we put it inside a function and call it whenever needed.

#### Simple Definition:

A function is a reusable block of code used to perform a specific task.

### 2.2 Why Do We Use Functions?

- To reduce code repetition
- To make programs easy to understand
- To divide a large program into small parts
- To reuse code multiple times

### 1.3 Defining and Using Functions

A function is created using the def keyword.

#### Syntax:

```
def function_name():  
    # code
```

#### Example:

```
def add():  
    a = 5  
    b = 3  
    print(a + b)  
add()
```

#### Explanation:

- def → used to define a function
- add → function name
- This function adds two numbers.
- add(): Calling the function

## 1.4 Functions with Arguments (Parameters)

Arguments are values passed to a function to perform operations.

### Example:

```
def add(a, b):  
    print(a + b)  
add(10, 20)
```

### Here:

- a and b are arguments
- Values are given when calling the function

## 1.5 Functions with Return Values

A function can return a result using the return keyword.

### Example:

```
def add(a, b):  
    return a + b  
result = add(5, 2)  
print(result)
```

**Output:** 7

## 1.6 Types of Functions in Python

Built-in and User-defined are types of functions.

**(1.) Built-in Functions:** These are already available in Python. No need to create them.

### Examples:

- print()
- len()
- sum()
- type()
- range()

**Example:**

```
numbers = [1, 2, 3, 4]
print(len(numbers))
```

**(2.) User-defined Functions:** These are created by the programmer using def.

**Example:**

```
def square(n):
    return n * n
print(square(4))
```

### 1.7 Types of User-defined Functions (Based on Arguments & Return)

1. No arguments, No return value

```
def show():
    print("Hello")
```

2. Arguments, No return value

```
def add(a, b):
    print(a + b)
```

3. No arguments, With return value

```
def get_number():
    return 10
```

4. Arguments, With return value

```
def add(a, b):
    return a + b
```

## 1.8 Advantages of Functions

- Code reusability
- Easy debugging
- Better program structure
- Saves time and effort

# OOP Concepts in Python

## 2.1 What is OOP?

OOP (Object-Oriented Programming) is a programming approach that uses objects and classes to design programs. It helps in organizing code in a structured and reusable way.

### Definition:

Object-Oriented Programming (OOP) is a programming method based on classes and objects to model real-world entities.

## 2.2 Class and Object

**1. Class:** A class is a blueprint or template used to create objects.

### Example:

```
class Student:  
    name = "Rahul"
```

**2.Object:** An object is an instance of a class.

### Example:

```
s1 = Student()  
print(s1.name)
```

## 2.3 Four Main OOP Concepts

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

### 1. What is Encapsulation?

Encapsulation is one of the core concepts of Object-Oriented Programming (OOP). It means binding data (variables) and methods (functions) together into a single unit called a class.

In encapsulation, the internal details of an object are hidden, and only necessary information is exposed.

### Definition:

Encapsulation is the process of wrapping data and methods together into a single unit and restricting direct access to data.

## 1.1 Why Do We Need Encapsulation?

Encapsulation is used to:

- Protect data from unauthorized access
- Control how data is modified
- Improve security
- Make programs easy to manage and maintain

## 1.2 Encapsulation in Python

In Python, encapsulation is achieved using classes and access specifiers.

A class contains:

- Data members (variables)
- Member functions (methods)

### Example Without Encapsulation (Problem)

```
class Student:
    name = "Rahul"
    marks = 90

s = Student()
s.marks = 10 # Anyone can change data
print(s.marks)
```

#### Problem:

- Data is not protected
- Anyone can modify it directly

### Example With Encapsulation

```
class Student:
    def __init__(self):
        self.name = "Rahul"
        self.__marks = 90 # private variable
    def show_marks(self):
        print(self.__marks)

s = Student()
s.show_marks()
```

#### Here:

- `__marks` is private
- It cannot be accessed directly
- Access is controlled using a method



## Access Specifiers in Encapsulation

Python supports three types of access specifiers:

### 1) Public Members

- Accessible from anywhere  
`self.name = "Rahul"`

### 2) Protected Members

- Accessible within the class and its child classes  
`self._age = 20`

### 3) Private Members

- Accessible only within the class  
`self.__marks = 90`

## Advantages of Encapsulation

- Data security
- Controlled access
- Better code organization
- Easy maintenance
- Improves reliability

## 2.What is Inheritance?

Inheritance is an OOP concept where one class acquires the properties and methods of another class.

- The class whose properties are inherited is called Parent class / Base class
- The class that inherits is called Child class / Derived class

### Definition:

Inheritance is the mechanism by which one class derives the properties and methods of another class.

### 2.1 Why Do We Use Inheritance?

Inheritance helps to:

- Reuse existing code
- Reduce duplication
- Improve program structure
- Make programs easy to maintain

## Real-Life Example of Inheritance

Vehicle → Car → Electric Car

- Vehicle: speed, fuel
- Car: wheels, engine
- Electric Car: battery

Each level inherits features from the previous one.

**Inheritance in Python:** Inheritance is implemented using classes.

### Syntax:

```
class ChildClass(ParentClass):  
    # code
```

### Simple Example of Inheritance

```
class Person:  
    def show(self):  
        print("This is a Person")  
  
class Student(Person):  
    pass  
  
s = Student()  
s.show()
```

### Explanation:

- Student inherits from Person
- show() method is used by Student without defining it again

### Example with Parent and Child Functions

```
class Person:  
    def details(self):  
        print("I am a person")  
  
class Student(Person):  
    def course(self):  
        print("I am a BTech student")  
  
s = Student()  
s.details()  
s.course()
```

## Types of Inheritance in Python

**1) Single Inheritance:** One child inherits from one parent.

```
class A:  
    pass  
class B(A):  
    pass
```

**2) Multilevel Inheritance:** A class inherits from a class which itself is inherited.

```
class A:  
    pass  
class B(A):  
    pass  
class C(B):  
    pass
```

**3) Multiple Inheritance:** A class inherits from more than one parent.

```
class A:  
    def showA(self):  
        print("Class A")  
class B:  
    def showB(self):  
        print("Class B")  
class C(A, B):  
    pass
```

```
obj = C()  
obj.showA()  
obj.showB()
```

**Method Overriding:** Child class can redefine a method of the parent class.

```
class Person:  
    def role(self):  
        print("Person")  
class Student(Person):  
    def role(self):  
        print("Student")
```

```
s = Student()  
s.role()
```

## Advantages of Inheritance

- Code reusability
- Easy maintenance
- Better code organization
- Faster development

## 3.What is Polymorphism?

Polymorphism is an important concept in Object-Oriented Programming (OOP). The word *Polymorphism* means "many forms".

It means the same function or method can perform different tasks depending on the situation.

### Definition:

Polymorphism is the ability of a function, method, or operator to behave differently for different objects or inputs.

### Real-Life Example

Person Example:

- A person can be a:
  - Teacher in school
  - Father at home
  - Customer in a shop

Same person → different roles → many forms

This is called polymorphism.

### Polymorphism in Python

In Python, polymorphism can be achieved in different ways:

- Function Polymorphism
- Operator Polymorphism
- Method Overriding (Runtime Polymorphism)

**Function Polymorphism:** The same function works with different types of data.

```
print(len("Python"))  
print(len([1, 2, 3, 4]))
```

### Explanation:

- len() works for string and list
- Same function → different behavior

**Operator Polymorphism:** Operators behave differently based on input.

```
print(5 + 3)      # Adds numbers
print("Hello " + "Students") # Joins strings
```

**Explanation:**

- + adds numbers
- + joins strings

Same operator → different work

**Method Overriding**

When a child class defines the same method as the parent class, it is called method overriding.

```
class Person:
    def show(self):
        print("I am a Person")

class Student(Person):
    def show(self):
        print("I am a Student")

s = Student()
s.show()
```

**Output:**

I am a Student

**Explanation:**

- Parent and child both have show() method
- Child method overrides parent method

**Another Simple Example**

```
class Animal:
    def sound(self):
        print("Animal makes sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

class Cat(Animal):
    def sound(self):
        print("Cat meows")
```

```
d = Dog()
c = Cat()
d.sound()
c.sound()
```

**Explanation:**

- Same method name sound()
- Different outputs for different objects

**Advantages of Polymorphism**

- Increases code flexibility
- Improves readability
- Supports method overriding
- Helps in code reusability

**4.What is Abstraction?**

Abstraction is one of the core concepts of Object-Oriented Programming (OOP). It means hiding the internal implementation details and showing only the essential features to the user.

The user knows what the object does, not how it does it.

**Definition:**

Abstraction is the process of hiding internal details and showing only the required functionality to the user.

**Why Do We Need Abstraction?**

Abstraction helps to:

- Reduce complexity
- Improve security
- Make programs easy to understand
- Focus on functionality rather than implementation
- Separate *what to do* from *how to do*

**Real-Life Examples of Abstraction**

Example 1: ATM Machine

- You insert card and enter PIN
- You can withdraw money
- You do NOT know how the bank server works

## Internal processing is hidden → Abstraction

### Example 2: Mobile Phone

- You press the call button
- You don't know signal processing

**Abstraction in Python:** In Python, abstraction is mainly achieved using:

- Classes
- Abstract Classes (conceptually)
- Methods

Python does not force abstraction, but we can design programs to achieve it.

### Simple Example (Without Abstraction Problem)

```
class Car:
    def start_engine(self):
        print("Checking engine")
        print("Starting engine")
        print("Car started")
c = Car()
c.start_engine()
```

#### Problem:

- User sees all internal steps
- Code looks complex

### Example Using Abstraction

```
class Car:
    def start(self):
        print("Car started")

c = Car()
c.start()
```

#### Explanation:

- User only uses start()
- Internal engine logic is hidden
- Simple and clean code

## Abstraction Using Methods (Important)

```
class BankAccount:  
    def withdraw(self, amount):  
        print("Amount withdrawn:", amount)  
  
account = BankAccount()  
account.withdraw(500)
```

### Explanation:

- User calls withdraw()
- Balance checking, validation is hidden
- Only required feature is visible

## Difference Between Abstraction and Encapsulation

| Abstraction                    | Encapsulation                    |
|--------------------------------|----------------------------------|
| Hides implementation details   | Hides data                       |
| Focus on what to do            | Focus on how data is protected   |
| Achieved using abstract design | Achieved using access specifiers |

## Advantages of Abstraction

- Reduces program complexity
- Improves code readability
- Increases security
- Makes code easy to modify
- Helps in large projects



## Exception Handling in Python

### 3.1 What is an Exception?

An exception is an error that occurs during the execution of a program.

When an error happens, Python stops the program and shows an error message. To prevent the program from crashing, we use Exception Handling.

### 3.2 Why Do We Need Exception Handling?

#### Without exception handling:

- Program stops suddenly
- Remaining code will not execute

#### With exception handling:

- Program continues running
- Errors are handled properly
- User-friendly messages can be shown

### 3.3 Common Types of Exceptions

| Exception Type    | Description                          |
|-------------------|--------------------------------------|
| ZeroDivisionError | Dividing a number by zero            |
| ValueError        | Wrong data type input                |
| TypeError         | Operation on incompatible data types |
| IndexError        | Invalid index in list                |
| KeyError          | Invalid key in dictionary            |
| FileNotFoundError | File does not exist                  |

### 3.4 try and except Block: Used to handle errors safely.

#### Syntax:

```
try:
    # code that may cause error
except:
    # code to handle error
```

### Example 1: ZeroDivisionError

```
try:
    a = int(input("Enter a number: "))
    b = int(input("Enter another number: "))
    print("Result:", a / b)

except ZeroDivisionError:
    print("You cannot divide by zero!")
```

**Note:** If user enters 0, program will not crash.

### 3.5 Handling Multiple Exceptions

```
try:
    num = int(input("Enter a number: "))
    print(10 / num)

except ZeroDivisionError:
    print("Cannot divide by zero")

except ValueError:
    print("Please enter a valid number")
```

### 3.6 Using else Block: The else block runs if no error occurs.

```
try:
    num = int(input("Enter number: "))

except ValueError:
    print("Invalid input")
else:
    print("You entered:", num)
```

### 3.6 finally Block: The finally block always runs, whether there is an error or not.

```
try:
    print(10 / 2)
except:
    print("Error occurred")
finally:
    print("This will always execute")
```

### 3.7 Raising an Exception: We can create our own error using raise.

```
age = int(input("Enter age: "))
```

```
if age < 18:
```

```
    raise ValueError("Age must be 18 or above")
```

# File Handling in Python

## 4.1 What is File Handling?

File handling allows us to:

- Create files
- Read files
- Write files
- Update files
- Delete files

Python uses the `open()` function to work with files.

## 4.2 Syntax of `open()`

```
file = open("filename", "mode")
```

## 4.3 File Modes

| Mode | Meaning                 |
|------|-------------------------|
| r    | Read (default)          |
| w    | Write (overwrites file) |
| a    | Append (adds data)      |
| x    | Create new file         |
| rb   | Read binary             |
| wb   | Write binary            |

## 4.4 Reading a File

**Example:**

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

#### 4.5 read() vs readline() vs readlines()

| Method      | Description               |
|-------------|---------------------------|
| read()      | Reads full file           |
| readline()  | Reads one line            |
| readlines() | Reads all lines into list |

#### 4.6 Writing to a File

##### Example:

```
file = open("data.txt", "w")
file.write("Hello Students")
file.close()
```

#### 4.7 Append to File

```
file = open("data.txt", "a")
file.write("\nNew Line Added")
file.close()
```

#### 4.8 Handling File Exceptions

```
try:
    with open("data.txt", "r") as file:
        print(file.read())
except FileNotFoundError:
    print("File not found!")
```

#### 4.9 Difference Between Error and Exception

| Error                     | Exception                 |
|---------------------------|---------------------------|
| Syntax mistake            | Runtime problem           |
| Detected before execution | Detected during execution |
| Cannot be handled         | Can be handled            |

\*\*\*\*\*

## **Unit 3: Modules and Packages**

### **Topic: Working with Modules and Packages in Python**

---

#### **1. Introduction**

In Python, large programs are divided into small, manageable parts to make them easy to write, understand, and maintain. These parts are called modules and packages.

#### **2. What is a Module?**

##### **Definition**

A module is a file that contains Python code such as functions, variables, and classes, which can be reused in other programs.

##### **Simple Definition:**

A module is a Python file that stores related code so we can use it again in other programs.

##### **Examples of Built-in Modules**

- math – for mathematical operations
- random – for generating random numbers
- datetime – for working with date and time
- sys – for system-related functions

#### **3. Creating a User-Defined Module**

##### **Steps**

1. Create a Python file (example: mymodule.py)
2. Write functions or variables in it
3. Import it into another Python program

## Example

File: mymodule.py

```
def greet(name):  
    return "Hello, " + name
```

Main Program

```
import mymodule  
print(mymodule.greet("Student"))
```

## 4. Ways to Import a Module

### 1. Import the Whole Module

```
import math  
print(math.sqrt(25))
```

### 2. Import Specific Functions

```
from math import sqrt  
print(sqrt(16))
```

### 3. Import with Alias

```
import math as m  
print(m.pi)
```

## 5. What is a Package?

### Definition

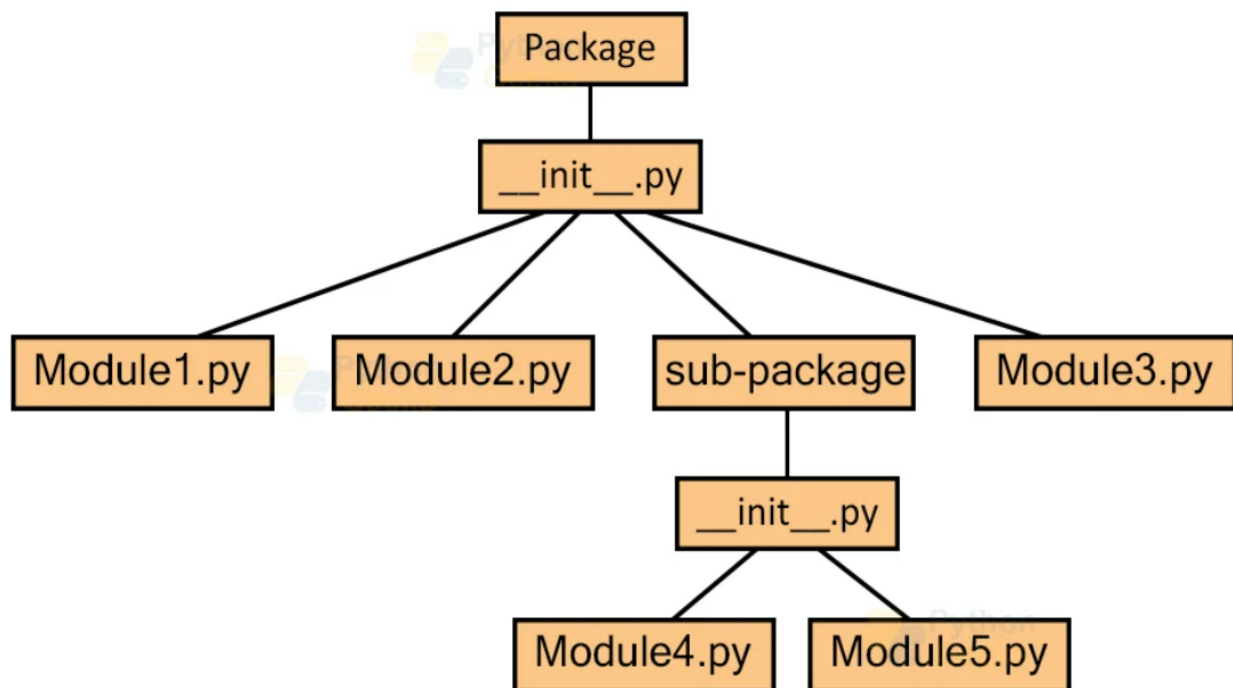
A package is a collection of related modules stored inside a folder (directory) to organize large Python programs.

### Simple Definition

A package is a folder that contains multiple Python files (modules) to keep big programs well organized.

## 6. Structure of a Package

### Structure of Packages



## 7. Using a Package

### Import a Module from a Package

```
from my_package import module1  
module1.function_name()
```



## Import a Specific Function

```
from my_package.module1 import function_name  
function_name()
```

## 8. Why Use Modules and Packages?

- Organizes large programs
- Improves code readability
- Avoids name conflicts
- Supports teamwork
- Makes debugging and maintenance easy
- Encourages code reusability

## 9. Difference Between Module and Package

| Feature    | Module               | Package                              |
|------------|----------------------|--------------------------------------|
| Definition | A single Python file | A folder containing multiple modules |
| Extension  | .py file             | Directory (folder)                   |
| Size       | Small program        | Large program                        |
| Example    | math.py              | mypackage/                           |

## **10. Built-in vs User-Defined**

### **Built-in Modules**

These are already available in Python.

Examples:

- math
- os
- sys
- random

### **User-Defined Modules**

These are created by the programmer.

Examples:

- student.py
- calculator.py

# Introduction to Popular Python Libraries for Specific Tasks

Topic: Python Libraries for Data Analysis, Web Development, and Game Development

---

## 1. What is a Python Library?

A **Python library** is a collection of pre-written code (modules and functions) that helps programmers perform common tasks easily and efficiently without writing everything from scratch.

### Simple Definition (Student Friendly)

A Python library is a **toolbox** that contains ready-made programs to help us do specific work like analyzing data, building websites, or creating games.

### Why Do We Use Libraries?

- Saves time
- Reduces coding effort
- Improves program quality
- Makes development faster
- Helps in professional projects

## 2. Categories of Popular Python Libraries

Python libraries are used in different fields. In this topic, we focus on three main areas:

1. Data Analysis
2. Web Development
3. Game Development

### 3. Python Libraries for Data Analysis

Data analysis means collecting, cleaning, and studying data to find useful information and patterns.

#### 3.1 NumPy

**Full Form:** Numerical Python

##### Definition

NumPy is a Python library used for working with **numbers, arrays, and mathematical operations**.

##### Features

- Supports large arrays and matrices
- Performs fast mathematical calculations
- Used in scientific computing

##### Example Use

- Calculating averages
- Working with lists of numbers
- Performing matrix operations
- 

##### Sample Code

```
import numpy as np  
arr = np.array([10, 20, 30, 40])  
print(arr)
```

## 3.2 Pandas

### Definition

Pandas is a Python library used for handling and analyzing structured data such as tables and files.

### Features

- Works with Excel and CSV files
- Supports data cleaning
- Allows filtering and sorting data

### Example Use

- Student marks analysis
- Attendance records
- Sales data processing

### Sample Code

```
import pandas as pd
data = {
    'Name': ['Amit', 'Riya', 'Suman'],
    'Marks': [85, 90, 78]
}
df = pd.DataFrame(data)
print(df)
```

## 3.3 Matplotlib

### Definition

Matplotlib is a Python library used to draw graphs and charts.

### Features

- Creates line charts, bar charts, and pie charts
- Helps in data visualization
- Makes reports more understandable

### Example Use

- Showing exam performance in graphs
- Visualizing sales growth

### Sample Code

```
import matplotlib.pyplot as plt  
x = [1, 2, 3, 4]  
y = [10, 20, 25, 30]  
plt.plot(x, y)  
plt.show()
```

## 4. Python Libraries for Web Development

Web development means **creating websites and web applications using Python.**

### 4.1 Flask

#### Definition

Flask is a lightweight Python library used to **build simple and small web applications.**

#### Features

- Easy to learn
- Flexible
- Suitable for beginners

#### Example Use

```
from flask import Flask
app = Flask(__name__)
@app.route("/")
def home():
    return "Welcome to My Website"
app.run()
```

## 4.2 Django

### Definition

Django is a high-level Python library used to **develop large and secure web applications**.

### Features

- Built-in security
- Database support
- User authentication system

### Example Use

- Online shopping websites
- College management systems

## 5. Python Libraries for Game Development

Game development means **creating computer games using programming**.

### 5.1 Pygame

#### Definition

Pygame is a Python library used to **create 2D games and multimedia applications**.

#### Features

- Supports graphics and sound
- Keyboard and mouse input
- Easy for beginners



## Example Use

- Snake game
- Car racing game

## Sample Code

```
import pygame
pygame.init()
screen = pygame.display.set_mode((400, 300))
pygame.display.set_caption("My First Game")
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    pygame.quit()
```

# PyCharm IDE, Git Integration, PyTest, and Database Connectivity

Topic: Working with PyCharm IDE, Git, Testing, and Databases in Python

---

## 1. Introduction

Modern Python development is not only about writing code but also about **using professional tools** to manage, test, and connect programs to databases. In this topic, students will learn about:

- PyCharm IDE
- Git and Git Integration
- PyTest for testing
- Python connectivity with MySQL and MongoDB
- CRUD Operations

## 2. What is an IDE?

### Definition

An **IDE (Integrated Development Environment)** is a software application that provides tools to write, run, test, and debug programs in one place.

### Simple Definition

An IDE is a **software tool where we write and run our Python programs easily.**

### Features of an IDE

- Code editor
- Run and debug tools
- Error highlighting
- File and project management

### 3. PyCharm IDE

#### Definition

PyCharm is a popular Python IDE developed by **JetBrains** that helps programmers write, test, and debug Python programs efficiently.

#### Features of PyCharm

- Smart code completion
- Error detection
- Debugging tools
- Project structure management
- Git integration

#### Uses

- Developing Python applications
- Web development using Django and Flask
- Data science and testing projects

### 4. Introduction to Git

#### Definition

Git is a **version control system** used to track changes in code and manage teamwork in software development.

#### Simple Definition

Git helps programmers **save different versions of their programs and work together safely.**

#### Uses of Git

- Tracks code changes
- Helps in teamwork
- Allows rollback to old versions
- Stores projects online using GitHub

## 5. Git Integration with PyCharm

### Meaning

Git integration means **using Git features directly inside PyCharm** without using the command line.

### Common Git Operations in PyCharm

- Initialize Git repository
- Commit changes
- Push to GitHub
- Pull updates
- View file history

### Steps (Basic)

1. Open project in PyCharm
2. Go to **VCS → Enable Version Control Integration**
3. Select Git
4. Connect to GitHub
5. Commit and push changes

## 6. Introduction to PyTest

### Definition

PyTest is a Python testing framework used to **check whether programs work correctly**.

### Simple Definition

PyTest is a tool that helps us **test our Python code automatically**.

### Features

- Simple syntax
- Automatic test discovery
- Detailed test reports

### Example:

```
def add(a, b):  
    return a + b  
  
def test_add():  
    assert add(2, 3) == 5
```

## 7. Python Connectivity with Databases

### Meaning

Python can connect to databases to **store, retrieve, update, and delete data**.

## 8. MySQL Database Connectivity

### Definition

MySQL is a **relational database** used to store data in tables.

### Python Library Used

- mysql-connector-python

### Example Code

```
import mysql.connector  
conn = mysql.connector.connect(  
    host="localhost",  
    user="root",  
    password="password",  
    database="college"  
)  
  
cursor = conn.cursor()  
cursor.execute("SELECT * FROM students")
```

```
for row in cursor:
```

```
    print(row)
```

```
conn.close()
```

## 9. MongoDB Database Connectivity

### Definition

MongoDB is a **NoSQL database** that stores data in the form of documents instead of tables.

### Python Library Used

- pymongo

### Example Code

```
from pymongo import MongoClient
client = MongoClient("mongodb://localhost:27017/")
db = client["college"]
collection = db["students"]
for student in collection.find():
    print(student)
```

## 10. CRUD Operations

### Meaning

CRUD stands for:

- **C** – Create (Insert Data)
- **R** – Read (View Data)
- **U** – Update (Modify Data)
- **D** – Delete (Remove Data)

## 11. CRUD Example (MySQL – Simple)

### Create (Insert)

```
cursor.execute("INSERT INTO students VALUES (1, 'Amit')")  
conn.commit()
```

### Read (Select)

```
cursor.execute("SELECT * FROM students")
```

### Update

```
cursor.execute("UPDATE students SET name='Riya' WHERE id=1")  
conn.commit()
```

### Delete

```
cursor.execute("DELETE FROM students WHERE id=1")  
conn.commit()
```

\*\*\*\*\*

## Unit: 4 – Flask Framework

Prepared By: Prince Kumar *Assistant Professor, CSE*

---

This unit introduces the fundamentals of web development using the Flask framework. Students will learn how to build dynamic web applications using Python, including routing, templates, database connectivity, authentication, and deployment.

### Topics Covered

- Introduction to Flask and Web Development
- Installation and Virtual Environment Setup
- Routing and Application Configuration
- URL Building
- HTTP Methods
- Templates (Jinja2)
- Static and Media Files
- Form Handling
- Database Connectivity (SQLite3 and MySQL)
- Exception and Error Handling
- Flash Messages
- Email Integration
- Authentication and Authorization (Flask-Login)
- Deployment of Flask Application

\*\*\*\*\*



# 1. Introduction to Flask and Web Development

---

## Q. What is Web Development?

Ans. Web development means creating websites and web applications that run on the internet. It involves designing, building, and maintaining websites.

There are mainly two parts of web development:

- A. Frontend (Client-Side)
- B. Backend (Server Side)

## A. Frontend (Client-Side)

**Frontend** is the part of a website that users can see and interact with directly in their web browser (like Chrome, Edge, etc.).

It focuses on:

- Design
- Layout
- Colors and fonts
- Buttons and forms
- User experience

**Simple words:** Frontend = What users see and click.

## Technologies Used in Frontend

### HTML (Hypertext Markup Language)

- Creates the **structure** of the webpage.
- Used for headings, paragraphs, images, forms, buttons.

#### Example:

- Login form
- Registration form
- Headings and text

### CSS (Cascading Style Sheets)

- Adds **style and design** to the webpage.
- Controls colors, fonts, spacing, layout.

#### Example:

- Background color
- Button color
- Font size

## JavaScript

- Adds **interactivity** to the webpage.
- Makes the page dynamic.

### Example:

- Button click actions
- Form validation
- Showing/hiding content
- Dropdown menus

## B.Backend (Server-Side)

Backend is the part of a website that works behind the scenes. Users cannot see it, but it is very important for the website to function properly.

**Simple words:** Backend = The brain of the website.

It handles the main logic, processes user requests, and manages data.

### Main Functions of Backend

- Handles Application Logic
- Stores and Retrieves Data
- Manages User Authentication
- Connects Frontend with Database

### Technologies Used in Backend

1. **Python:** Used to write backend logic.

#### Frameworks:

- Flask
- Django

2. **Databases:** Used to store data permanently.

#### Examples:

- MySQL
- MongoDB

## Q. What is Flask?

Ans. Flask is a web framework written in Python that helps developers build web applications easily.

It allows us to connect:

-  Web pages (HTML)
-  Python code
-  Databases

So basically, Flask helps create the backend of a website.

## Q. Why is Flask called Lightweight?

Ans. Flask is called a lightweight (micro) framework because:

- It is simple
- It has fewer built-in features
- It gives developers freedom to add only what they need

## Key Points Explained

### 1. Written in Python

Flask is built using the Python programming language, so if you know Python, you can easily learn Flask.

### 2. Simple and Beginner-Friendly

- Easy to understand
- Clean and short code
- Perfect for students and new developers

### 3. Used for Backend Development

Backend means the server-side part of a website.

Backend handles:

- Login verification
- Saving data
- Connecting to database
- Processing forms

Flask helps us do all this.

#### 4. Connects Web Pages with Python Code

When a user clicks a button or submits a form:

- Flask receives the request
- Python processes the data
- Flask sends a response back

👉 This makes the website interactive.

#### Examples of Flask Usage

Flask is used to build:

- 🗝️ Login systems (username & password verification)
- 🎓 Student portals
- 📄 Online forms
- 🔗 REST APIs (used by mobile apps & websites to share data)

#### Q. What is a Framework?

Ans. A framework is a ready-made structure that helps developers build applications quickly and easily.

Instead of writing everything from the beginning, a framework provides:

- Pre-written code
- Built-in tools
- Proper structure
- Rules to follow

👉 This saves time, effort, and reduces errors.

#### Real-Life Example

- 🏠 **Building a house from scratch**
  - You design everything yourself
  - Takes more time
  - More difficult
- 🏡 **Using a ready-made design**
  - Structure is already planned
  - Faster construction
  - Easier work

👉 A framework is like a ready house design for software development.

## 2.Installation and Virtual Environment Setup

### 1. Install Python

Before using Flask, you must install Python.

Steps:

1. Go to the official Python website.
2. Download the latest version.
3. Install it.
4. During installation, ✓ check “Add Python to PATH”.

To verify installation, open Command Prompt / Terminal and type: `python --version`  
If Python is installed correctly, it will show the version number.

### 2. What is a Virtual Environment?

A virtual environment is a separate workspace for your project.

It allows you to:

- Install project-specific libraries
- Avoid conflicts between different projects
- Keep your system clean

👉 Each project can have its own packages and versions.

### 3. Create a Virtual Environment

**Step 1:** Open terminal / command prompt

Go to your project folder.

**Step 2:** Create virtual environment

`python -m venv venv`

Here:

- `venv` = name of the virtual environment folder

### 4. Activate the Virtual Environment

- On Windows: `venv\Scripts\activate`
- On Mac/Linux: `source venv/bin/activate`
- After activation, you will see: `(venv)` in front of your terminal line. That means the virtual environment is active.

### 5. Install Flask

After activating the virtual environment, install Flask: `pip install flask`

To check installation: `pip list`

You will see Flask in the list.

## 6. Why Use Virtual Environment?

### ❖ Without virtual environment:

- Packages may conflict
- Different projects may break
- System becomes messy

### ❖ With virtual environment:

- Clean project
- No conflicts
- Easy management

### 3. Routing and Application Configuration

#### Q. What is Routing?

**Ans.** Routing means connecting a URL (web address) to a function in the program.  
In simple words:

👉 When a user types a URL, Flask decides which function should run.

#### Example of Routing:

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Welcome to Home Page"

@app.route("/about")
def about():
    return "This is About Page"

if __name__ == "__main__":
    app.run(debug=True)
```

#### Explanation:

- / → Calls the home() function
- /about → Calls the about() function
- @app.route() → Defines the URL

#### Q. What is Application Configuration?

Application configuration means **setting rules and settings** for the Flask application.  
It controls:

- Debug mode
- Security key
- Database settings
- Application behavior

#### Example of Configuration

```
app.config["DEBUG"] = True
app.config["SECRET_KEY"] = "mysecretkey"
```

## 4.URL Building, HTTP Methods, Templates & Static Files

### Q. What is URL Building?

Ans. URL building means generating URLs dynamically using the `url_for()` function. Instead of writing URL paths manually, Flask creates them automatically.

### Q. Why use `url_for()`?

- Avoid hardcoding URLs
- If route changes, no need to update everywhere
- Makes code clean and maintainable

### Syntax:

```
url_for('function_name')
```

### Example:

```
from flask import Flask, url_for
```

```
app = Flask(__name__)
```

```
@app.route('/')
```

```
def home():
```

```
    return "Home Page"
```

```
@app.route('/about')
```

```
def about():
```

```
    return url_for('home')
```

### Q. What are HTTP Methods?

Ans. HTTP methods define what action the client wants to perform.

### Common HTTP Methods:

| Method | Purpose                 |
|--------|-------------------------|
| GET    | To get data from server |
| POST   | To send data to server  |
| PUT    | To update data          |
| DELETE | To delete data          |



### Example (GET & POST):

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    return "Login Page"
```

### Q. What are Templates?

Ans. Templates allow us to write HTML with dynamic data.  
Flask uses Jinja2 template engine.

### Q. Why Templates?

- Separate Python code from HTML.
- Display dynamic data.
- Example: app.py

```
from flask import Flask, render_template
app = Flask(__name__)

@app.route('/')
def home():
    return render_template('index.html', name="Prince")
```

- **index.html (inside templates folder)**

```
<h1>Hello {{ name }}</h1>
```

### Important Jinja2 Syntax:

#### Syntax

```
{{ variable }}
{% if %}
{% for %}
```

#### Use

Print variable  
Conditional  
Loop

### Q. What are Static Files?

Static files are files that do not change:

- CSS
- JavaScript
- Images

## 5. Form Handling, Database Connectivity (SQLite3 & MySQL), Exception and Error Handling, Flash Messages, and Email Integration.

---

### Q. What is Form Handling?

Form handling means collecting data from users (like login form, registration form) and processing it in the backend.

### Steps in Form Handling

1. Create HTML Form
2. Use POST method
3. Get data using request.form
4. Process data in Flask route

### Example

#### HTML Form (login.html)

```
<form method="POST">
    Name: <input type="text" name="username">
    <input type="submit">
</form>
```

### Flask Code

```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        name = request.form['username']
        return "Welcome " + name
    return render_template('login.html')

if __name__ == "__main__":
    app.run(debug=True)
```

### Points

- request.form → Used to get form data
- POST → Used to send data securely
- Always specify methods=['GET','POST']

## 2. Database Connectivity in Flask

Flask supports many databases like:

- SQLite3 (Lightweight, built-in)
- MySQL (Large applications)

### A) SQLite3 in Flask

What is SQLite?

- Small database
- No separate server required
- Stored in a file

#### Example (SQLite)

```
import sqlite3

conn = sqlite3.connect('student.db')
cursor = conn.cursor()

cursor.execute("CREATE TABLE IF NOT EXISTS students(id INTEGER
PRIMARY KEY, name TEXT)")
cursor.execute("INSERT INTO students(name) VALUES('Prince')")
conn.commit()
conn.close()
```

### B) MySQL in Flask

To connect MySQL:

Install: pip install mysql-connector-python

Example:

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="root",
    password="1234",
    database="testdb"
)

cursor = db.cursor()
cursor.execute("SELECT * FROM students")
```

## Exam Difference Table

| Feature         | SQLite            | MySQL               |
|-----------------|-------------------|---------------------|
| Server Required | No                | Yes                 |
| Best For        | Small Apps        | Large Apps          |
| Speed           | Fast (small data) | Better for big data |

## Exception and Error Handling in Flask

### Q.What is Exception Handling?

Ans.Handling errors properly so that the application does not crash.

### Python Exception Example

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

### Flask Error Handling

```
Custom Error Page:
@app.errorhandler(404)
def page_not_found(error):
    return "Page Not Found", 404
```

### Common Errors

- 404 → Page Not Found
- 500 → Internal Server Error

## Flash Messages in Flask

### Q. What is Flash Message?

Flash messages show temporary messages to users.

Example:

- Login successful
- Invalid password
- Registration completed

### Steps to Use Flash

1. Set secret key
2. Use flash()
3. Display in template

### Example

```
from flask import Flask, flash, render_template

app = Flask(__name__)
app.secret_key = "secret"

@app.route('/')
def home():
    flash("Login Successful!")
    return render_template('home.html')
```

### In HTML

```
{% with messages = get_flashed_messages() %}
    {% for message in messages %}
        <p>{{ message }}</p>
    {% endfor %}
{% endwith %}
```

### Important for Exam

- Flash requires secret\_key
- Used for one-time messages

### Email Integration in Flask

Flask can send emails using SMTP.

### Install Flask-Mail

pip install flask-mail

### Example

```
from flask import Flask
from flask_mail import Mail, Message

app = Flask(__name__)

app.config['MAIL_SERVER'] = 'smtp.gmail.com'
app.config['MAIL_PORT'] = 587
app.config['MAIL_USE_TLS'] = True
app.config['MAIL_USERNAME'] = 'your_email@gmail.com'
app.config['MAIL_PASSWORD'] = 'your_password'

mail = Mail(app)
```

```
@app.route('/send')
def send_mail():
    msg = Message("Hello",
        sender="your_email@gmail.com",
        recipients=["student@gmail.com"])
    msg.body = "This is a test email"
    mail.send(msg)
    return "Email Sent!"
```

### Q. What is Authentication?

Ans. Authentication means verifying the identity of a user.

#### Example:

- Login with username & password
- OTP verification

If credentials are correct → User is authenticated.

### Q. What is Authorization?

Ans. Authorization means checking what the user is allowed to access.

#### Example:

- Admin can delete data
- Student can only view data

### Q. Difference Between Authentication & Authorization

| Authentication | Authorization        |
|----------------|----------------------|
| Who are you?   | What can you access? |
| Login process  | Permission control   |
| Happens first  | Happens after login  |

### Q. What is Flask-Login?

Flask-Login is an extension that helps manage:

- User login
- User logout
- User session
- Remember logged-in users

It handles authentication easily.

## Install Flask-Login

```
pip install flask-login
```

## Basic Steps in Flask-Login

1. Import Flask-Login
2. Create LoginManager
3. Create User class
4. Define user\_loader
5. Use login\_user()
6. Use logout\_user()
7. Protect routes using @login\_required

## Example

```
from flask import Flask, render_template
from flask_login import LoginManager, UserMixin, login_user, login_required, logout_user

app = Flask(__name__)
app.secret_key = "secretkey"

login_manager = LoginManager()
login_manager.init_app(app)

# Dummy User Class
class User(UserMixin):
    def __init__(self, id):
        self.id = id

@login_manager.user_loader
def load_user(user_id):
    return User(user_id)

@app.route('/login')
def login():
    user = User(1)
    login_user(user)
    return "User Logged In"

@app.route('/dashboard')
@login_required
def dashboard():
```

```

    return "Welcome to Dashboard"
@app.route('/logout')
def logout():
    logout_user()
    return "Logged Out"

if __name__ == "__main__":
    app.run(debug=True)

```

## Important Functions

| Function         | Purpose                     |
|------------------|-----------------------------|
| login_user(user) | Logs in user                |
| logout_user()    | Logs out user               |
| @login_required  | Protects route              |
| current_user     | Gets current logged-in user |

## Points

- Flask-Login manages sessions
- Requires secret key
- Used for secure web applications
- Helps implement role-based access control

## Q. What is Deployment?

Ans. Deployment means making your Flask application available online so users can access it from anywhere.

## Development vs Deployment

| Development           | Deployment     |
|-----------------------|----------------|
| Runs on local machine | Runs on server |
| Debug mode ON         | Debug mode OFF |
| Not secure            | Secure         |



## Steps for Deployment

1. Turn off debug mode
2. Use Production Server (Gunicorn / uWSGI)
3. Choose hosting platform
4. Upload project
5. Configure environment variables

## Common Deployment Platforms

- PythonAnywhere
- Heroku
- AWS
- Render
- Railway

## Example: Running in Production

Instead of:

```
app.run(debug=True)
```

Use:

```
app.run()
```

Or use Gunicorn:

```
pip install gunicorn  
gunicorn app:app
```

## Files Required for Deployment

- app.py
- requirements.txt
- Procfile (for some platforms)
- templates folder
- static folder

## Important Environment Variables

- SECRET\_KEY
- DATABASE\_URL
- MAIL\_USERNAME
- MAIL\_PASSWORD

## Points

- Deployment makes app public
- Debug mode should be OFF
- Production server is required
- Hosting platforms are used

\*\*\*\*\*

## Unit 5

### Django Framework

---

#### 1. Introduction to Django Framework

##### What is Django?

Django is a high-level open-source web framework written in Python that allows developers to build secure, scalable, and maintainable web applications quickly.

Django follows the Model–View–Template (MVT) architecture, which separates the application into different components to make development easier and organized.

It was developed by Adrian Holovaty and Simon Willison in 2005 while working at the Lawrence Journal-World to manage news websites efficiently.

##### Definition

Django is a Python-based web framework that provides built-in tools and libraries to develop dynamic web applications quickly and efficiently.

##### Features of Django

###### 1. Rapid Development

Django provides many built-in modules which reduce development time.

Example:

- Authentication system
- Admin panel
- Form handling

###### 2. Security

Django protects applications from common web attacks such as:

- SQL Injection
- Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF)

###### 3. Object Relational Mapping (ORM)

ORM allows developers to interact with the database using **Python code instead of SQL queries**.

Example:

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
```

Here, the **Student model automatically creates a database table**.

#### 4. Scalability

Django can handle large and complex applications with heavy traffic.

Example:

Large websites like **Instagram** and **Pinterest** use Django.

#### 5. Built-in Admin Interface

Django automatically generates an **admin panel** where administrators can manage data easily.

Example:

- Add students
- Delete records
- Update information

#### Advantages of Django

- Fast development
- Secure framework
- Clean and maintainable code
- Built-in authentication system
- Easy database integration
- Large community support

#### Applications of Django

Django is used to build:

- E-commerce websites
- Social media platforms
- Educational websites
- Blogging systems
- Content management systems

Example:

An **online student management system** where teachers can manage student data.

## 2. Django Project Installation in Virtual Environment

### What is a Virtual Environment?

A **Virtual Environment** is an isolated Python environment where project-specific packages are installed separately from the global Python installation.

### Why Virtual Environment is Used?

1. Avoid package conflicts
2. Maintain different versions of libraries
3. Manage project dependencies easily

Example:

One project may use **Django 3**, while another project may use **Django 5**.

## Steps to Install Django in Virtual Environment

### Step 1: Install Virtual Environment

```
pip install virtualenv
```

### Step 2: Create Virtual Environment

```
python -m venv myenv
```

#### Example:

```
python -m venv django_env
```

- Here **django\_env** is the name of the environment.

### Step 3: Activate Virtual Environment

#### For Windows:

```
django_env\Scripts\activate
```

After activation, the environment name appears before the command prompt.

#### Example:

```
(django_env) C:\project>
```

### Step 4: Install Django

```
pip install django
```

### Step 5: Verify Installation

```
django-admin --version
```

#### Example Output: 5.0

- This confirms Django is successfully installed.

## 3. Phases in Django Project Creation

Developing a Django project involves several phases.

### Phase 1: Environment Setup

In this phase:

- Install Python
- Install Django
- Create Virtual Environment
- Activate Virtual Environment

#### Example:

```
python -m venv myenv
```

```
pip install Django
```

## Phase 2: Create Django Project

A Django project is created using the command: `django-admin startproject projectname`

### Example:

```
django-admin startproject college
```

- This command creates the project folder structure.

## Phase 3: Create Application

In Django, a **project can contain multiple applications**.

### Example:

A college project may contain:

- student app
- teacher app
- attendance app

**Command:** `python manage.py startapp student`

## Phase 4: Define Models

Models define the structure of the database table.

### Example:

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    course = models.CharField(max_length=50)
    age = models.IntegerField()
```

## Phase 5: Migration

Migration converts models into database tables.

### Commands:

```
python manage.py makemigrations
python manage.py migrate
```

**Example:** This creates a **Student table in the database**.

## Phase 6: Create Views

Views contain the logic of the application.

### Example:

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to Django Website")
```

## Phase 7: Configure URLs

URLs connect user requests to views.

### Example:

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.home)
]
```

## Phase 8: Create Templates

Templates are HTML files used to display web pages.

### Example:

```
<html>
    <head>
        <title>Home Page</title>
    </head>
    <body>

        <h1>Welcome to Django</h1>

    </body>
</html>
```

## 4. Create a Django Project

### Definition

A **Django project** is a collection of configuration files and applications used to build a complete web application.

### Creation of App in Django

#### What is a Django App?

In Django, an **application (app)** is a module that performs a specific function within a project. A **Django project can contain multiple apps**, and each app handles a particular feature of the website.

### Example

Consider a **College Management System** project.

It may contain different apps such as:

- **Student App** – manages student information
- **Teacher App** – manages teacher data
- **Attendance App** – manages attendance records

Each app works independently but is part of the same project.

## Steps to Create an App in Django

### Step 1: Navigate to the Project Directory

Open the terminal or command prompt and go to your project folder.

**Example:**

```
cd myproject
```

### Step 2: Create an App

Use the following command:

```
python manage.py startapp appname
```

**Example:**

```
python manage.py startapp student
```

- Here **student** is the name of the application.

### Step 3: Register the App in settings.py

After creating the app, it must be **registered in the project settings**.

Open **settings.py** and add the app name in **INSTALLED\_APPS**.

**Example:**

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'student',  
]
```

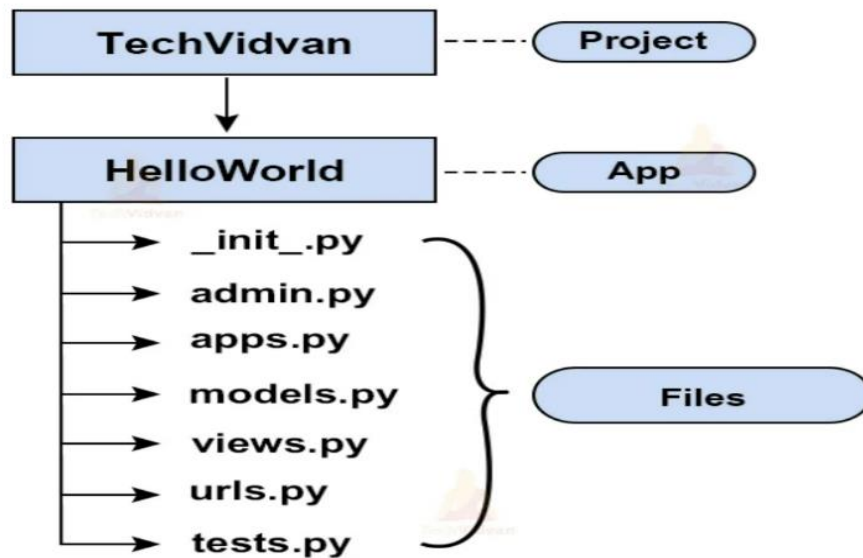
- This allows Django to recognize and use the app.

## 5. Structure of a Django App

After creating the app, Django automatically generates several files.

**Example structure:**





## Explanation of App Files

### 1. models.py

This file is used to create **database models (tables)**.

#### Example:

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
```

This creates a **Student table in the database**.

### 2. views.py

Views contain the **application logic** and control what the user sees.

#### Example:

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to Student App")
```

### 3.admin.py

This file is used to **register models in the admin panel**.

#### Example:

```
from django.contrib import admin
from .models import Student

admin.site.register(Student)
```

#### 4. apps.py

Contains configuration settings for the application.

##### Example:

```
from django.apps import AppConfig

class StudentConfig(AppConfig):
    name = 'student'
```

#### 5. migrations folder

This folder stores **database migration files**. Migration files track changes made to models.

##### Example commands:

```
python manage.py makemigrations
python manage.py migrate
```

#### 6. tests.py

Used to write **test cases** for the application.

##### Example:

```
from django.test import TestCase
```

### 6. Working with Django Admin Console

#### What is Django Admin Console?

The **Admin Console** is a built-in feature of Django that provides a **web-based interface for managing database records**.

It allows administrators to:

- Add data
- Update records
- Delete records
- Manage users

without writing additional code.

#### Steps to Use the Admin Console

##### Step 1: Create a Superuser

To access the admin console, create an admin account.

##### Command:

```
python manage.py createsuperuser
```

### Example Input:

Username: admin  
Email: admin@gmail.com  
Password: \*\*\*\*\*

### Step 2: Run the Server

Start the Django development server.

```
python manage.py runserver
```

### Step 3: Open the Admin Panel

Open the browser and go to:

```
http://127.0.0.1:8000/admin
```

Login using the **superuser username and password**.

### Registering Models in Admin Panel

To display models in the admin console, they must be **registered in admin.py**.

#### Example:

```
from django.contrib import admin
from .models import Student

admin.site.register(Student)
```

After registering, the **Student model will appear in the admin dashboard**.

### Example: Working with Admin Console

#### Step 1: Create Model

```
from django.db import models
class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
```

#### Step 2: Register Model in Admin

```
from django.contrib import admin
from .models import Student

admin.site.register(Student)
```

#### Step 3: Run Migration

```
python manage.py makemigrations
python manage.py migrate
```

## Step 4: Add Data Through Admin Panel

Steps:

1. Open **admin panel**
2. Click **Student**
3. Click **Add Student**
4. Enter data
5. Click **Save**

## Advantages of Django Admin Console

1. Easy data management
2. No need to write extra code
3. Automatically generated interface
4. Secure authentication system
5. Quick data updates

## 7. Creating Views in Django

### What is a View?

In Django, a **View** is a Python function or class that receives a web request and returns a web response. Views contain the **business logic of the application** and determine what data will be displayed to the user.

### Definition

A **View** is a Python function that takes a request object as an argument and returns a response such as an HTML page, JSON data, or a simple message.

### Example of a View

```
Open views.py
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to Django Website")
```

### Explanation

- **request** → HTTP request from user
- **HttpResponse()** → sends response to browser

## Types of Views

### 1. Function-Based Views

These are simple Python functions.

**Example:**

```
def about(request):
    return HttpResponse("About Page")
```

### 2. Class-Based Views

These use Python classes to create views.

### Example:

```
from django.views import View
from django.http import HttpResponse

class HomeView(View):
    def get(self, request):
        return HttpResponse("Welcome")
```

## 8.URL Mapping in Django

### What is URL Mapping?

URL Mapping connects a **URL address with a view function**. When a user enters a URL in the browser, Django checks the **urls.py** file and calls the corresponding view.

### Example

Open **urls.py**

```
from django.urls import path
from . import views
```

```
urlpatterns = [
    path("", views.home),
    path('about/', views.about),
]
```

### Explanation

| URL    | View       |
|--------|------------|
| /      | home view  |
| /about | about view |

### Example URL:

http://127.0.0.1:8000/about

This will call the **about()** view.

## Template System in Django

### What is a Template?

A **Template** is an HTML file used to display data dynamically. Django templates allow developers to insert **dynamic content into HTML pages**.

## Template Folder Structure

Create a folder named:

```
templates
```

Example structure:

```
project/
  templates/
    home.html
```

## Example Template

home.html

```
<html>
```

```
<head>
```

```
  <title>Home Page</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Welcome to Django Website</h1>
```

```
</body>
```

```
</html>
```

## Rendering Template in View

Open **views.py**

```
from django.shortcuts import render
```

```
def home(request):
    return render(request, "home.html")
```

## Passing Data to Template

**Example:**

```
def home(request):
    data = {"name": "Rahul"}
    return render(request, "home.html", data)
```

**Template:**

```
<h1>Welcome {{name}}</h1>
```

**Output:**

```
Welcome Rahul
```

## 9. Working with Models

### What is a Model?

A **Model** represents a **database table** in Django.

Models define the **structure of data stored in the database**.

## Example Model

Open **models.py**

```
from django.db import models
```

```
class Student(models.Model):  
    name = models.CharField(max_length=100)  
    age = models.IntegerField()  
    course = models.CharField(max_length=50)
```

## Model Fields

| Field         | Description   |
|---------------|---------------|
| CharField     | Text field    |
| IntegerField  | Numeric field |
| DateTimeField | Date value    |
| EmailField    | Email address |

## Migration Commands

After creating models run:

```
python manage.py makemigrations  
python manage.py migrate
```

These commands create tables in the database.

## 10. Form Processing in Django

### What is Form Processing?

Form processing is used to **collect data from users through HTML forms and store it in the database.**

Example:

- Registration form
- Login form
- Feedback form

### Example HTML Form

```
<form method="POST">  
  
    <input type="text" name="name">  
  
    <input type="submit">  
  
</form>
```

## Handling Form in Views

```
from django.shortcuts import render

def form_view(request):

    if request.method == "POST":
        name = request.POST.get("name")
        print(name)

    return render(request, "form.html")
```

### Explanation

- **request.POST** → retrieves form data
- **POST method** → used to send data securely

## 11. Static and Media Files

### Static Files

Static files are **files that do not change**.

#### Examples:

- CSS
- JavaScript
- Images

### Media Files

Media files are **user uploaded files**.

#### Examples:

- profile pictures
- documents
- videos

### Example model for media file:

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    photo = models.ImageField(upload_to='images/')
```

Uploaded images are stored in the **media folder**.

## 12. Django App Deployment

### What is Deployment?

Deployment is the process of **making a Django application available on the internet** so users can access it.

#### Example:

Instead of running locally:

`http://127.0.0.1:8000`

The website becomes accessible online.



## Steps for Deployment

1. Set **DEBUG = False** in settings.py
2. Configure **ALLOWED\_HOSTS**
3. Collect static files

## Example:

`python manage.py collectstatic`

4. Use a web server like:
  - **Nginx**
  - **Gunicorn**
5. Host the project on a cloud server.

## Popular Platforms for Deployment

Examples:

- **Amazon Web Services**
- **Heroku**
- **DigitalOcean**

\*\*\*\*\*

## Unit 6

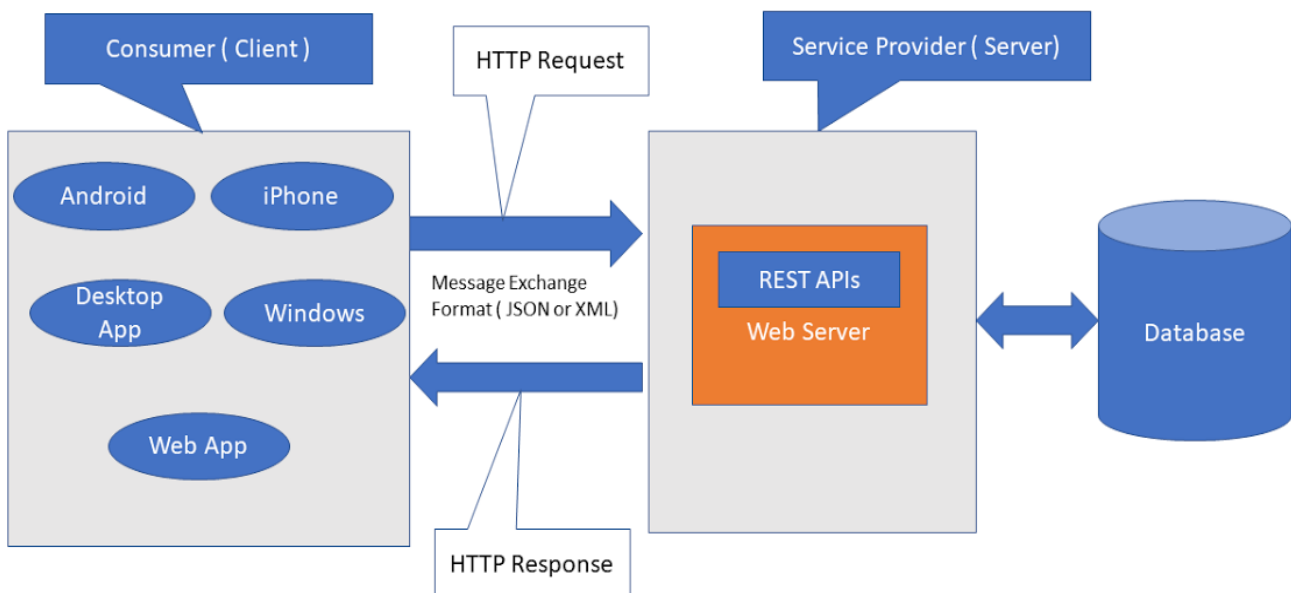
### \*\*\*RESTful API\*\*\*

#### 1. Introduction to RESTful API

REST (Representational State Transfer) is an architectural style used for designing network-based applications. It was introduced by Roy Fielding in 2000.

A RESTful API is an API that follows REST principles and uses HTTP protocol to enable communication between client and server.

#### REST – Architecture



#### 2. Features of REST Architecture

- a) Stateless – Each request is independent
- b) Client-Server Architecture – Separation of frontend and backend
- c) Cacheable – Responses can be stored for reuse
- d) Uniform Interface – Standard way of communication
- e) Layered System – Multiple layers between client and server

#### 3. HTTP Protocol and Its Role in RESTful APIs

##### HTTP (HyperText Transfer Protocol):

HTTP is a communication protocol used to transfer data between client and server.

## 4. Working of HTTP

1. Client sends request
2. Server processes request
3. Server sends response

## 5. Role of HTTP in REST APIs

- REST APIs use HTTP methods to perform operations
- HTTP helps in:
  - Sending requests
  - Receiving responses
  - Transferring data
  - Providing status codes

## 6. HTTP Methods in RESTful APIs

HTTP methods are used to perform operations on resources.

| Method | Description          |
|--------|----------------------|
| GET    | Retrieve data        |
| POST   | Create new data      |
| PUT    | Update existing data |
| DELETE | Delete data          |

### Example:

GET /students → Get all students  
POST /students → Add new student  
PUT /students/1 → Update student  
DELETE /students/1 → Delete student

### Real-Life Example

In an online shopping system:

- GET → View products
- POST → Place order
- PUT → Update address
- DELETE → Cancel order

## 7. URLs and Resource Representation

**Resource:** A resource is any data such as student, product, or user.

## URL (Uniform Resource Locator)

URL is used to identify resources.

### Examples

/students

/students/1

/products/10

- /students → All students
- /students/1 → Specific student

## 8. Resource Representation

Data is transferred in formats like:

- JSON (most common)
- XML

### Example (JSON)

```
{  
    "id": 1,  
    "name": "Prince",  
    "course": "CSE"  
}
```

## 9. Best Practices for RESTful API Design

### 1. Use Proper HTTP Methods

- GET → Retrieve data
- POST → Create data
- PUT → Update data
- DELETE → Remove data

### 2. Use Meaningful URLs

✓ Good:

/students

/orders/5

### 3. Use HTTP Status Codes

| Code | Meaning      |
|------|--------------|
| 200  | OK           |
| 201  | Created      |
| 400  | Bad Request  |
| 404  | Not Found    |
| 500  | Server Error |

#### 4. Use JSON Format

- Lightweight
- Easy to understand

#### 5. Keep API Stateless

- Each request must contain all information
- Server should not store client state

#### 6. Use Proper Naming Convention

- Use nouns instead of verbs

✓ /students

✗ /getStudents

#### 10. Advantages of RESTful APIs

- Simple and easy to use
- Scalable
- Fast performance
- Platform independent

#### 11. Disadvantages

- Stateless (no memory)
- Security issues
- May require multiple requests

#### 12. API Versioning

##### Definition

API Versioning is the process of managing changes in an API without affecting existing users. It allows developers to introduce new features while maintaining compatibility with older versions.

##### Need for API Versioning

- To avoid breaking existing client applications
- To support multiple versions simultaneously
- To allow gradual migration to newer versions
- To maintain backward compatibility

##### Types of API Versioning

###### 1. URI Versioning (Most Common)

Version is included in the URL.

**Example:**

/api/v1/students

/api/v2/students

**Header Versioning**

Version is specified in HTTP headers.

**Example:**

Accept: application/vnd.api.v1+json

**2. Query Parameter Versioning**

Version is passed in query string.

**Example:**

/students?version=1

**13. Error Handling in RESTful APIs****Definition**

Error handling is the process of identifying, managing, and returning meaningful error responses to the client when something goes wrong.

**Importance**

- Helps client understand the issue
- Improves debugging
- Enhances user experience

**14. Common HTTP Status Codes**

| Code | Meaning               |
|------|-----------------------|
| 200  | OK (Success)          |
| 201  | Created               |
| 400  | Bad Request           |
| 401  | Unauthorized          |
| 403  | Forbidden             |
| 404  | Not Found             |
| 500  | Internal Server Error |

## Standard Error Response Format

```
{  
    "error": "Student not found",  
    "status": 404,  
    "message": "No student exists with given ID"  
}
```

## 15. Consuming RESTful APIs

### Definition

Consuming an API means using it to send requests and receive responses from a server.

### 1. Using cURL

#### What is cURL?

cURL is a command-line tool used to send HTTP requests.

#### Uses

- Testing APIs
- Debugging
- Automation

### 2. Using Postman

#### What is Postman?

Postman is a GUI-based tool used for testing APIs.

#### Features

- Easy to use interface
- Supports all HTTP methods
- Shows response in structured format

#### Steps to Use

1. Select method (GET/POST)
2. Enter API URL
3. Click Send
4. View response

## 16. Using Python Requests Library

### What is Requests Library?

A Python library used to send HTTP requests easily.

### Example

```
import requests
response = requests.get("https://api.example.com/students")
print(response.json())
```

### Advantages

- Simple syntax
- Easy integration in applications
- Supports all HTTP methods

## 17. Building RESTful APIs using Frameworks

### Using Flask

#### Definition

Flask is a lightweight Python web framework used to build REST APIs.

#### Example

```
from flask import Flask, jsonify

app = Flask(__name__)
@app.route('/students', methods=['GET'])
def get_students():
    return jsonify({"students": ["A", "B"]})

app.run()
```

### Advantages

- Simple and easy
- Flexible
- Good for beginners

### Using FastAPI

#### Definition

FastAPI is a modern, high-performance Python framework for building APIs.

#### Example

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/students")
def get_students():
    return {"students": ["A", "B"]}
```



## Advantages

- Very fast
- Automatic API documentation
- Supports async programming

## Flask vs FastAPI

| Feature       | Flask    | FastAPI   |
|---------------|----------|-----------|
| Speed         | Moderate | Very Fast |
| Learning      | Easy     | Moderate  |
| Documentation | Manual   | Automatic |

## 18. Building Scalable RESTful APIs

### Definition

Scalability is the ability of a system to handle increased load efficiently.

### Techniques for Scalability

- Load balancing
- Caching responses
- Database optimization
- Microservices architecture

## Securing RESTful APIs

### Need for Security

- Protect user data
- Prevent unauthorized access

### Common Security Methods

- Authentication (Login system)
- Authorization (Access control)
- HTTPS (Secure communication)
- API keys and tokens

\*\*\*\*\*

## **Important Questions**

1. Define RESTful API.
2. Explain REST Architecture Principles.
3. Explain HTTP Methods with Example.
4. What is Resource and URL in REST API?
5. Explain HTTP Status Codes.
6. Explain API Versioning.
7. Explain Error Handling in REST API.
8. Explain API Consumption Tools
9. Explain Flask and FastAPI
10. Write Advantages of REST API
11. What is Scalability in REST API?
12. Explain Security in REST APIs